

C PROGRAMMING

Unit-I: Introductory Concepts & C Fundamentals

Comprehensive Study Notes

Topic 1: Types of Programming Languages

1.1 Classification of Programming Languages

Programming languages are categorized into different levels based on their abstraction from machine code.

1.2 Classification Table

Level	Language Type	Example	Translator	Speed
1GL	Machine Language	10110100	None	Fastest
2GL	Assembly Language	MOV AX, 5	Assembler	Very Fast
3GL	High-Level Language	printf("Hello")	Compiler/Interpreter	Fast
4GL	Query Language	SELECT * FROM	Interpreter	Slow

1.3 Machine Language (1GL)

Machine language is the lowest-level programming language. Programs are written in **binary code (0s and 1s)** that the CPU can execute directly without any translation.

Feature	Description
Form	Binary code (0s and 1s)
Execution	Directly understood by CPU
Portability	Not portable (machine-specific)
Speed	Fastest
Ease	Very difficult to understand

```
10110100 00000001 ; Load 1
10110000 00000001 ; Add 1
11110000 00000010 ; Halt
```

1.4 Assembly Language (2GL)

Assembly language uses **mnemonics** (symbolic codes) like MOV, ADD, SUB to represent machine instructions. An assembler converts assembly code to machine code.

Feature	Description
Form	Mnemonics (symbols and abbreviations)
Execution	Assembler converts to machine code
Portability	Not portable
Speed	Very fast
Ease	Easier than machine language

```
MOV AX, 1 ; Move 1 to AX
ADD AX, 1 ; Add 1 to AX
HLT      ; Halt
```

1.5 High-Level Language (3GL)

High-level languages use **English-like statements** and are machine-independent. Examples include C, C++, Java, Python, FORTRAN, COBOL.

Feature	Description
Form	English-like statements
Execution	Compiler/Interpreter converts to machine code
Portability	Highly portable

Speed	Slower than low-level
Ease	Easy to learn and use

Examples:

- **C:** System programming
- **C++:** Object-oriented programming
- **Java:** Cross-platform development
- **Python:** General purpose, AI

Advantages: Easy to learn, portable, faster development, large library support.

Disadvantages: Slower than machine/assembly, uses more memory, less hardware control.

1.6 Fourth Generation Language (4GL)

4GL languages use **natural-like statements** for database queries and report generation.

Examples: SQL, Report Generators, Application Generators.

1.7 Based on Execution

Type	Description	Examples
Compiled	Source code converted to machine code before execution	C, C++
Interpreted	Source code executed line by line	Python, BASIC
Hybrid	Compiled to bytecode, then interpreted	Java

Topic 2: Introduction to C

2.1 History of C

C was developed by **Dennis Ritchie** at Bell Labs in 1972. It was created to write the UNIX operating system.

Year	Event
1969-1973	Developed by Dennis Ritchie at Bell Labs
1978	First edition of "The C Programming Language" by Kernighan and Ritchie
1989	ANSI C standard (C89)
1999	C99 standard
2011	C11 standard

2.2 Features of C

Feature	Description
Simple	Small set of keywords, easy syntax
Structured	Supports functions and structured programming
Efficient	Fast execution, low memory usage
Portable	Code runs on different machines with minor changes
Rich Library	Many built-in functions
Low-Level Access	Direct memory manipulation via pointers

Modular	Programs can be divided into functions
Recursion	Supports recursive function calls

2.3 Structure of a C Program

```
// Documentation section
#include <stdio.h> // Preprocessor directive

// Global declaration section
int globalVar = 10;

// Function declaration (prototype)
int add(int a, int b);

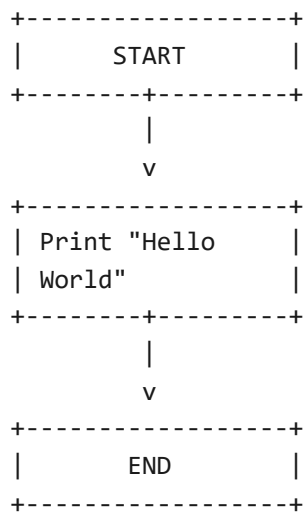
// main() function - program entry point
int main()
{
    int x = 5, y = 3, result;
    result = add(x, y);
    printf("Sum = %d\n", result);
    return 0;
}

// Function definition
int add(int a, int b)
{
    return a + b;
}
```

Topic 3: Simple C Programs

3.1 Program: Hello World

Flowchart:



Code:

```
#include <stdio.h>    // Standard Input Output header file

int main()            // Main function - program starts here
{
    printf("Hello, World!\n"); // Print statement
    return 0;          // Return 0 to indicate successful execution
}
```

Output:

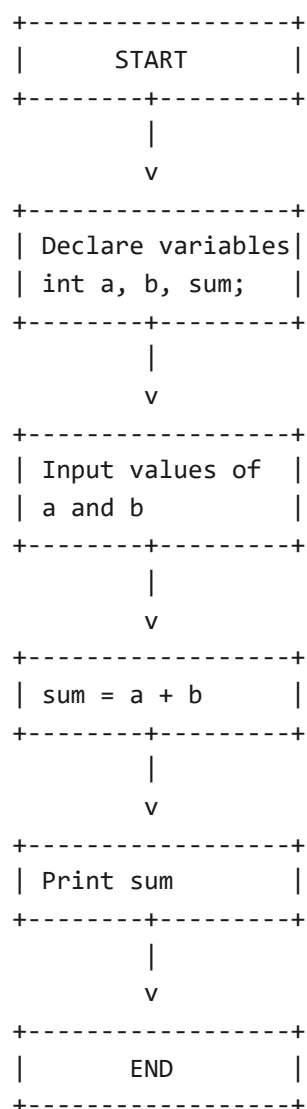
```
Hello, World!
```

Explanation:

- `#include <stdio.h>` - Includes standard input/output library
- `int main()` - Main function, entry point of program
- `printf()` - Function to print output to console
- `\n` - Newline character
- `return 0;` - Indicates program executed successfully

3.2 Program: Addition of Two Numbers

Flowchart:



Code:

```
#include <stdio.h>

int main()
{
    int a, b, sum;        // Variable declaration

    printf("Enter first number: ");
    scanf("%d", &a);      // Input first number

    printf("Enter second number: ");
    scanf("%d", &b);      // Input second number

    sum = a + b;          // Addition
    printf("Sum = %d\n", sum); // Output result

    return 0;
}
```

Output:

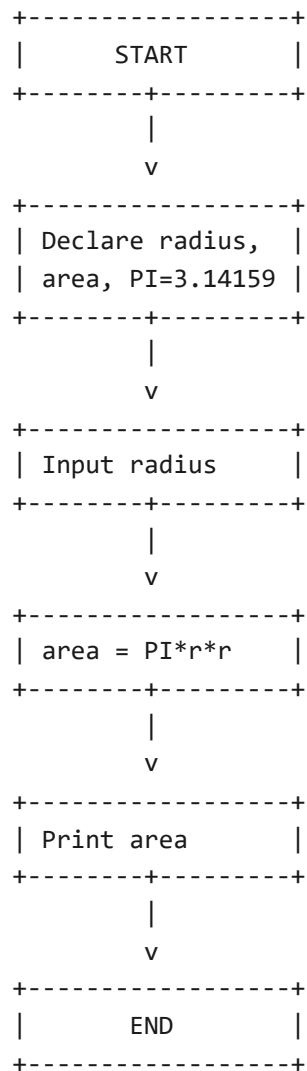
```
Enter first number: 10
Enter second number: 20
Sum = 30
```

Explanation:

- `int a, b, sum;` - Declaring integer variables
- `scanf("%d", &a);` - Reading integer input from user
- `&a` - Address of variable (using address operator)
- `%d` - Format specifier for integer
- `sum = a + b;` - Arithmetic operation

3.3 Program: Calculate Area of Circle

Flowchart:



Code:

```
#include <stdio.h>
#define PI 3.14159      // Symbolic constant

int main()
{
    float radius, area;
```

```
printf("Enter radius of circle: ");
scanf("%f", &radius);

area = PI * radius * radius;

printf("Area of circle = %.2f\n", area);

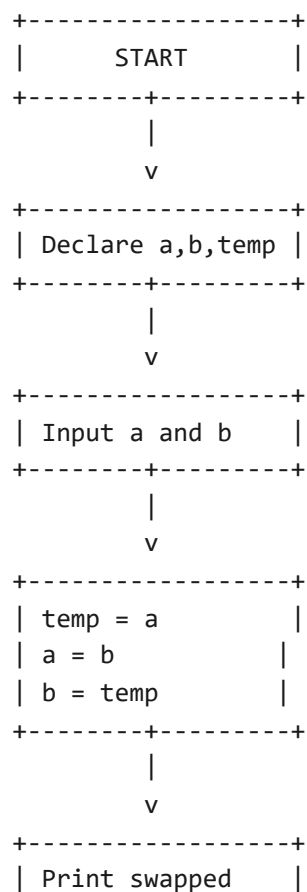
return 0;
}
```

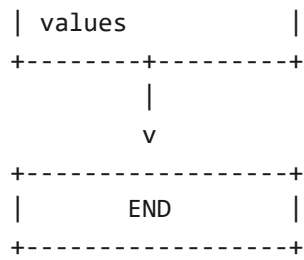
Output:

```
Enter radius of circle: 5
Area of circle = 78.54
```

3.4 Program: Swap Two Numbers

Flowchart:





Code:

```
#include <stdio.h>

int main()
{
    int a, b, temp;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("Before swapping: a = %d, b = %d\n", a, b);

    // Swapping logic
    temp = a;
    a = b;
    b = temp;

    printf("After swapping: a = %d, b = %d\n", a, b);

    return 0;
}
```

Output:

```
Enter two numbers: 10 20
Before swapping: a = 10, b = 20
After swapping: a = 20, b = 10
```

Topic 4: Desirable Program Characteristics

4.1 Program Characteristics

Characteristic	Description
Reliability	Program produces correct results consistently
Readability	Easy to understand and read
Efficiency	Uses minimal time and memory
Maintainability	Easy to modify and update
Portability	Can run on different systems
User-Friendly	Easy to use and interact with
Robustness	Handles errors gracefully
Modularity	Divided into independent modules

4.2 Good Programming Practices

Practice	Description
Comments	Add comments to explain code
Meaningful Names	Use descriptive variable names
Indentation	Proper indentation for readability
Modular Code	Break code into functions
Error Handling	Check for errors and handle them
Code Reusability	Write reusable code

Consistency

Follow consistent style

Topic 5: C Fundamentals

5.1 C Character Set

Category	Characters
Alphabets	A-Z, a-z
Digits	0-9
Special Symbols	+ - * / = < > () { } [] . , ; : " ' & ! ~ # %
White Spaces	Space, Tab, Newline, Carriage Return
Escape Sequences	\n, \t, \r, \a, \b, \f, \\, \', \"

5.2 Escape Sequences

Sequence	Description
\n	Newline (Line feed)
\t	Horizontal Tab
\r	Carriage Return
\a	Alert (Bell)
\b	Backspace
\f	Form Feed
\\	Backslash
\'	Single Quote
\"	Double Quote

<code>\0</code>	Null Character
<code>\xhh</code>	Hexadecimal Number

5.3 Identifiers

Definition: Identifiers are names given to variables, functions, arrays, and other user-defined items.

Rule	Description	Valid	Invalid
1	Must start with letter or underscore	<code>_count</code>	<code>2count</code>
2	Can contain letters, digits, underscore	<code>my_var</code>	<code>my-var</code>
3	Cannot contain special characters	<code>sum123</code>	<code>sum@123</code>
4	Cannot be a keyword	<code>total</code>	<code>int</code>
5	Case-sensitive	<code>Sum</code> and <code>sum</code>	-

5.4 Keywords

Definition: Keywords are reserved words with special meaning in C. They cannot be used as identifiers.

<code>auto</code>	<code>double</code>	<code>int</code>	<code>struct</code>	<code>break</code>	<code>else</code>
<code>long</code>	<code>switch</code>	<code>case</code>	<code>enum</code>	<code>register</code>	<code>typedef</code>
<code>char</code>	<code>extern</code>	<code>return</code>	<code>union</code>	<code>const</code>	<code>float</code>
<code>short</code>	<code>unsigned</code>	<code>continue</code>	<code>for</code>	<code>signed</code>	<code>void</code>
<code>default</code>	<code>goto</code>	<code>sizeof</code>	<code>volatile</code>	<code>do</code>	<code>if</code>
<code>static</code>	<code>while</code>				

5.5 Data Types

Definition: Data types specify the type of data that a variable can store.

Data Types Table

Data Type	Size (Bytes)	Range	Format Specifier
char	1	-128 to 127	%c
unsigned char	1	0 to 255	%c
int	2 or 4	-32768 to 32767 or -2^{31} to $2^{31}-1$	%d
unsigned int	2 or 4	0 to 65535	%u
short	2	-32768 to 32767	%hd
long	4	-2^{31} to $2^{31}-1$	%ld
float	4	$3.4e-38$ to $3.4e+38$	%f
double	8	$1.7e-308$ to $1.7e+308$	%lf
void	0	No value	-

5.6 Constants

Definition: Constants are fixed values that do not change during program execution.

Types of Constants:

Type	Example	Description
Integer Constant	10, -5, 0	Whole numbers

Octal Constant	012, 045	Starts with 0
Hexadecimal Constant	0x1A, 0xFF	Starts with 0x
Float Constant	3.14, -0.5	Decimal numbers
Character Constant	'A', '5', '*'	Enclosed in single quotes
String Constant	"Hello"	Enclosed in double quotes
Escape Sequence	'\n', '\t'	Special characters

5.7 Variables

Definition: Variables are named memory locations that store values which can change during program execution.

Rule	Description
1	Must be declared before use
2	Can be initialized at declaration
3	Can change value during execution
4	Follow identifier naming rules

```
int age;           // Declaration
int count = 10;    // Declaration with initialization
float pi = 3.14159; // Floating point variable
char grade = 'A';  // Character variable
```

5.8 Arrays

Definition: An array is a collection of similar data elements stored in contiguous memory locations.

```
// Array Declaration
data_type array_name[size];

// Array Initialization
int numbers[5] = {10, 20, 30, 40, 50};
char name[20] = "John";

// Accessing Array Elements
numbers[0] = 10; // First element
numbers[4] = 50; // Fifth element
```

5.9 Declarations

Declaration	Example	Description
Variable	int a;	Declares an integer variable
Constant	const int b = 10;	Declares a constant
Array	int arr[10];	Declares an array
Function	int add(int, int);	Declares a function
Pointer	int *ptr;	Declares a pointer

5.10 Expressions

Definition: An expression is a combination of operators, operands, and function calls that evaluates to a value.

Type	Example	Description
------	---------	-------------

Arithmetic	<code>a + b</code>	Arithmetic operations
Relational	<code>a > b</code>	Comparison operations
Logical	<code>a && b</code>	Logical operations
Assignment	<code>a = 10</code>	Assignment operations
Function Call	<code>add(5, 10)</code>	Function call expression

5.11 Statements

Definition: A statement is a complete instruction to the computer.

Statement	Example	Description
Declaration	<code>int a;</code>	Declares a variable
Assignment	<code>a = 5;</code>	Assigns a value
Compound	<code>{ a=5; b=10; }</code>	Block of statements
Selection	<code>if(a>b) {...}</code>	Conditional execution
Iteration	<code>for(i=0;i<10;i++)</code>	Loop execution
Jump	<code>break; continue; goto;</code>	Transfer control
Return	<code>return 0;</code>	Returns from function

5.12 Symbolic Constants

Definition: Symbolic constants are identifiers that represent constant values.

```
#define PI 3.14159
#define MAX 100
#define STR "Hello"

int main()
{
    float area = PI * r * r;
```

```
int arr[MAX];  
printf(STR);  
return 0;  
}
```

Rule	Description
------	-------------

1	Defined using <code>#define</code> directive
2	No semicolon at end
3	Usually written in uppercase
4	Replaced by preprocessor before compilation
5	Cannot be changed during execution

Topic 6: HPU Important Questions

6.1 Short Answer Questions (2-5 Marks)

1. What are the types of programming languages?
2. What are the features of C language?
3. Explain the structure of a C program.
4. What is the C character set?
5. What are identifiers? Give rules for identifiers.
6. What are keywords? Give examples.
7. What are the different data types in C?
8. What are constants? Explain different types.
9. What are variables? How are they declared?
10. What is an array? How is it declared?
11. What is a symbolic constant? Give example.
12. What are escape sequences? Give examples.
13. What are desirable program characteristics?
14. Write a program to print "Hello World".

6.2 Long Answer Questions (10 Marks)

1. Explain different levels of programming languages.
2. Explain the features and structure of C language in detail.
3. Explain identifiers, keywords, and data types in C with examples.
4. Explain constants and variables in C with examples.
5. What is an array? How to declare and initialize arrays?
6. Explain symbolic constants and their usage.
7. Write a program to find sum and average of two numbers.
8. Write a program to find area of a rectangle.
9. Explain desirable program characteristics.

6.3 Objective Questions (1 Mark)

1. C was developed by: (a) Dennis Ritchie (b) Kernighan (c) Linus Torvalds (d) Bill Gates **Answer: (a)**
2. C language is a: (a) Machine language (b) Assembly language (c) High-level language (d) None **Answer: (c)**
3. The size of int is: (a) 1 byte (b) 2 bytes (c) 4 bytes (d) Depends on compiler **Answer: (d)**
4. Which is not a valid identifier? (a) _count (b) total (c) 2sum (d) value123 **Answer: (c)**
5. Escape sequence for newline is: (a) \t (b) \n (c) \r (d) \a **Answer: (b)**
6. Which is a keyword? (a) int (b) integer (c) real (d) floatvar **Answer: (a)**
7. The size of char is: (a) 1 byte (b) 2 bytes (c) 4 bytes (d) 8 bytes **Answer: (a)**
8. #define MAX 100 is a: (a) Macro (b) Constant (c) Variable (d) Function **Answer: (a)**

6.4 Program Writing Questions (5 Marks)

1. Write a program to print "Welcome to C Programming".
2. Write a program to find sum and product of two numbers.
3. Write a program to find area of a triangle.
4. Write a program to swap two numbers without using a third variable.
5. Write a program to find simple interest.

Quick Revision

Data Types Summary

Data Type	Size	Range
char	1	-128 to 127
int	2/4	-32768 to 32767
float	4	3.4e-38 to 3.4e+38
double	8	1.7e-308 to 1.7e+308
void	0	No value

Format Specifiers

Format Specifier	Data Type
%c	char
%d	int
%f	float
%lf	double
%s	string
%u	unsigned int

Program Structure

```
#include <stdio.h>    // Header file
#define PI 3.14159    // Symbolic constant
```

```
int main()           // Main function
{
    // Variable declarations
    // Statements
    return 0;        // Return
}
```

End of Unit-I Notes